목차

1. 정렬 / 이분 탐색 관련 1
1.1 정렬 비교 함수 구현 [PY] . . . 1
1.2 좌표 압축 [PY] . . . 1
1.3 이분 탐색과 선택 알고리즘 [PY] . . . 1

2. 그래프/트리 (Easy) 2
2.1 인접 리스트, DFS, BFS [PY] . . . 2
2.2 위상 정렬 [PY] . . . 2
2.3 최단 거리 - Floyd Warshall [PY] . . . 2
2.4 최단 거리 - Dijkstra [PY] . . . 3
2.5 최단 거리 - Bellman Ford [PY] . . . 3
2.6 유니온 파인드 + 최소 신장 트리(Kruskal) [PY] . . . 3

3. 자료구조 3
3.1 세그먼트 트리 [PY] . . . 3
3.2 세그먼트 트리 + 레이지 프로퍼게이션 [PY] . . . 4
3.3 Convex Hull Trick [PY] . . . 4
3.4 퍼시스턴트 세그먼트 트리 [PY] . . . 5

4. 그래프/트리 (Hard) 5
4.1 SCC - Kosaraju [PY] . . . 5
4.2 BCC - Tarjan [PY] . . . 6
4.3 최대 유량 - Dinic [PY] . . . 6
4.4 MCMF [PY] . . . 7
4.5 이분 매칭 - Hopcroft Karp [PY] . . . 7
4.6 최소 공통 조상(LCA) [PY] . . . 8
4.7 Heavy Light Decomposition [PY] . . . 8

5. 수학 9
5.1 나눗셈, 최대공약수, 최소공배수 [PY] . . . 9
5.2 빠른 거듭제곱 [PY] . . . 9
5.3 소수 판별, 소인수분해 [PY] . . . 9
5.4 에라토스테네스의 체, 소인수분해 [PY] . . . 9
5.5 선형 시간 체, 곱셈적 함수 전처리 [PY] . . . 10
5.6 확장 유클리드 알고리즘 [PY] . . . 10
5.7 중국인의 나머지 정리 [PY] . . . 10
5.8 이항 계수를 소수로 나눈 나머지 [PY] . . . 10
5.9 빠른 소수 판별, 소인수분해 - Miller Rabin, Pollard Rho [PY] . . . 10
5.10 가우스 소거법 - RREF, 랭크, 행렬식, 역행렬 [PY] . . . 11
5.11 다항식 곱셈(FFT) [PY] . . . 11

6. 문자열 13
6.1 문자열 해싱 [PY] . . . 13
6.2 문자열 매칭 - KMP [PY] . . . 13
6.3 가장 긴 팰린드롬 부분 문자열 - Manacher [PY] . . . 13
6.4 문자열 매칭 - Z [PY] . . . 13
6.5 접미사 배열 [PY] . . . 13

7. 계산 기하 14
7.1 2차원 계산 기하 템플릿 + CCW [PY] . . . 14
7.2 360도 각도 정렬 [PY] . . . 14
7.3 다각형 넓이 [PY] . . . 14
7.4 선분 교차 판정 [PY] . . . 14
7.5 다각형 내부 판별 [PY] . . . 15
7.6 볼록 껍질 - Monotone Chain [PY] . . . 15
7.7 가장 먼 두 점 - Rotating Calipers [PY] . . . 15
7.8 볼록 다각형 내부 판별 [PY] . . . 15

8. 기타 15
8.1 가장 긴 증가하는 부분 수열(LIS) [PY] . . . 15
8.2 이분 탐색 [PY] . . . 16
8.3 삼분 탐색 [PY] . . . 16
8.4 Python 랜덤, 순서 통계, 비트마스킹 [PY] . . . 16
8.5 빠른 입력(Fast Input from STDIN) [PY] . . . 17
8.6 구간별 약수 최대 개수, 최대 소수 . . . 17
8.7 카탈란 수, 심슨 적분, 그런디 정리, 픽의 정리, 페르마 포인트, 오일러 정리 . . . 17
8.8 경우의 수 - 포함 배제, 스털링 수, 벨 수 . . . 18
8.9 삼각형의 오심 - 외심, 내심, 무게중심, 수심, 방심 . . . 18
8.10 미적분, 뉴턴 랩슨법 . . . 19
8.11 문제 풀이 체크리스트 . . . 19

1. 정렬 / 이분 탐색 관련

1.1 정렬 비교 함수 구현 [PY]

```
# 안정 정렬 O(N log N). 같은 키의 원래 순서를 유지한다.
from functools import cmp_to_key

def sort_points(points):
    # x 오름차순, 같으면 y 오름차순.
    return sorted(points, key=lambda p: (p[0], p[1]))

def sort_with_comparator(values, compare):
    # 비교 함수는 a<b, a=b, a>b에 각각 음수·0·양수를 반환한다.
    return sorted(values, key=cmp_to_key(compare))
```

1.2 좌표 압축 [PY]

```
# 중복을 제거하고 정렬한 값에 0부터 번호를 붙인다.
def coordinate_compress(values):
    # (0부터 시작하는 압축 좌표, 복원용 정렬 고유 값). O(N log N), 공간 O(N).
    unique = sorted(set(values))
    rank = {value: i for i, value in enumerate(unique)}
    return [rank[value] for value in values], unique
```

1.3 이분 탐색과 선택 알고리즘 [PY]

```
# 정렬 배열의 경계 탐색 O(log N). 리스트 삽입은 O(N).
from bisect import bisect_left, bisect_right

def equal_range(values, x):
    # 정렬 배열에서 x가 나타나는 구간 [왼쪽, 오른쪽)을 구한다.
    return bisect_left(values, x), bisect_right(values, x)
```

```
def contains(values, x):
    i = bisect_left(values, x)
    return i < len(values) and values[i] == x

def nth_element(values, k):
    # 배열을 직접 바꾸어 k번째 값을 선택한다. 기대 O(N), 최악 O(N^2).
    from random import randrange
    if not 0 <= k < len(values):
        raise IndexError(k)
    l, r = 0, len(values)
    while r - l > 1:
        pivot = values[randrange(l, r)]
        a = i = l
        b = r
        while i < b:
            if values[i] < pivot:
                values[a], values[i] = values[i], values[a]
                a += 1
                i += 1
            elif values[i] > pivot:
                b -= 1
                values[b], values[i] = values[i], values[b]
            else:
                i += 1
        if k < a:
            r = a
        elif k >= b:
            l = b
        else:
            break
    return values[k]
```

2. 그래프/트리 (Easy)

2.1 인접 리스트, DFS, BFS [PY]

```
# 인접 리스트 순회 O(V+E). 정점 번호는 0부터 시작한다.
from collections import deque

from heapq import heappop, heappush

from math import inf

def bfs(graph, start):
    # 시작점에서의 간선 수 거리. 도달 불가는 -1.
    distance = [-1] * len(graph)
    distance[start] = 0
    queue = deque([start])
    while queue:
        v = queue.popleft()
        for u in graph[v]:
            if distance[u] == -1:
                distance[u] = distance[v] + 1
                queue.append(u)
    return distance

def dfs(graph, start):
    # 재귀 DFS와 같은 인접 리스트 순서로 방문. 재귀 깊이 제한 없음.
    seen = [False] * len(graph)
    seen[start] = True
    order = [start]
    stack = [iter(graph[start])]
    while stack:
        u = next(stack[-1], None)
        if u is None:
            stack.pop()
        elif not seen[u]:
            seen[u] = True
            order.append(u)
            stack.append(iter(graph[u]))
    return order
```

2.2 위상 정렬 [PY]

```
# 진입 차수가 0인 정점부터 꺼내 위상 순서를 구한다. O(V+E).
from collections import deque

from heapq import heappop, heappush

from math import inf

def topological_sort(graph):
    # DAG의 위상 순서. 사이클이 있으면 None. 빈 그래프는 [].
    indegree = [0] * len(graph)
    for neighbors in graph:
        for u in neighbors:
            indegree[u] += 1
    queue = deque(v for v, degree in enumerate(indegree) if degree == 0)
    order = []
    while queue:
        v = queue.popleft()
        order.append(v)
        for u in graph[v]:
            indegree[u] -= 1
            if indegree[u] == 0:
                queue.append(u)
    return order if len(order) == len(graph) else None
```

2.3 최단 거리 - Floyd Warshall [PY]

```
# 모든 쌍 최단 거리 O(V^3). d[v][v]<0이면 음수 사이클이 있다.
# 음수 사이클을 거치는 경로의 거리는 유효하지 않다.
from math import inf

def floyd_warshall(n, edges):
    d = [[inf] * n for _ in range(n)]
    for v in range(n):
        d[v][v] = 0
    for u, v, w in edges:
        d[u][v] = min(d[u][v], w)
    for k in range(n):
        for i in range(n):
            if d[i][k] == inf:
```

```
                continue
            for j in range(n):
                d[i][j] = min(d[i][j], d[i][k] + d[k][j])
    return d
```

2.4 최단 거리 - Dijkstra [PY]

```
# 음이 아닌 가중치의 최단 거리. 최소 힙으로 다음 정점을 고른다.
from collections import deque

from heapq import heappop, heappush

from math import inf

def dijkstra(graph, start):
    # 인접 항목은 (정점, 가중치). (거리, 부모) 반환; 도달 불가는 (inf,-1).
    # 음수 가중치는 허용하지 않는다.
    if any(weight < 0 for edges in graph for _, weight in edges):
        raise ValueError('Dijkstra requires nonnegative weights')
    distance = [inf] * len(graph)
    parent = [-1] * len(graph)
    distance[start] = 0
    queue = [(0, start)]
    while queue:
        cost, v = heappop(queue)
        if cost != distance[v]:
            continue
        for u, weight in graph[v]:
            candidate = cost + weight
            if candidate < distance[u]:
                distance[u] = candidate
                parent[u] = v
                heappush(queue, (candidate, u))
    return distance, parent

def restore_path(parent, start, target):
    # dijkstra의 부모 배열에서 경로 복원. 도달 불가면 []. O(V).
    path = []
    while target != -1:
        path.append(target)
        if target == start:
            return path[::-1]
        target = parent[target]
    return []
```

2.5 최단 거리 - Bellman Ford [PY]

```
# 간선 (u,v,w)를 반복 완화한다. O(VE). 도달 가능한 음수 사이클은 None.
from math import inf

def bellman_ford(n, edges, start):
    edges = list(edges)
    d = [inf] * n
    d[start] = 0
    for step in range(n):
        changed = False
        for u, v, w in edges:
            if d[u] != inf and d[v] > d[u] + w:
                d[v] = d[u] + w
                changed = True
        if not changed:
            return d
    return None
```

2.6 유니온 파인드 + 최소 신장 트리(Kruskal) [PY]

```
# 간선을 가중치순으로 골라 최소 신장 포리스트를 만든다. O(V+E log(E+1)).
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n
        self.components = n

    def find(self, v):
        while self.parent[v] != v:
            self.parent[v] = self.parent[self.parent[v]]
            v = self.parent[v]
        return v

    def union(self, u, v):
        u, v = self.find(u), self.find(v)
        if u == v:
            return False
        if self.size[u] < self.size[v]:
            u, v = v, u
        self.parent[v] = u
        self.size[u] += self.size[v]
        self.components -= 1
        return True

def kruskal(n, edges):
    # 간선은 (가중치,u,v). (비용, 선택 간선) 반환; 연결 그래프면 간선 수 n-1.
    dsu = DSU(n)
    total = 0
    selected = []
    for weight, u, v in sorted(edges):
        if dsu.union(u, v):
            total += weight
            selected.append((weight, u, v))
    return total, selected
```

3. 자료구조

3.1 세그먼트 트리 [PY]

```
# 점 대입과 구간 [l,r) 합 O(log N). 초기화 O(N).

class SegmentTree:
    def __init__(self, values):
        self.n = len(values)
        self.size = 1 << max(0, (self.n - 1).bit_length())
        self.tree = [0] * (2 * self.size)
        self.tree[self.size:self.size + self.n] = values
        for i in range(self.size - 1, 0, -1):
```

```
            self.tree[i] = self.tree[i * 2] + self.tree[i * 2 + 1]
    def set(self, i, value):
        if not 0 <= i < self.n:
            raise IndexError(i)
        i += self.size
        self.tree[i] = value
        while i > 1:
            i //= 2
            self.tree[i] = self.tree[i * 2] + self.tree[i * 2 + 1]
    def sum(self, l, r):
        if not 0 <= l <= r <= self.n:
            raise IndexError((l, r))
        answer = 0
        l += self.size
        r += self.size
        while l < r:
            if l & 1:
                answer += self.tree[l]
                l += 1
            if r & 1:
                r -= 1
                answer += self.tree[r]
            l //= 2
            r //= 2
        return answer
```

3.2 세그먼트 트리 + 레이지 프로퍼게이션 [PY]

```
# 구간 [l,r) 덧셈과 합 O(log N). 갱신을 자식에 지연 전파한다.

class LazySegmentTree:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (4 * max(n, 1))
        self.lazy = [0] * len(self.tree)
    def _apply(self, v, l, r, value):
        self.tree[v] += (r - l) * value
        self.lazy[v] += value
    def _push(self, v, l, r):
        m = (l + r) // 2
        self._apply(v * 2, l, m, self.lazy[v])
        self._apply(v * 2 + 1, m, r, self.lazy[v])
        self.lazy[v] = 0
    def _query(self, v, l, r, a, b, value):
        if b <= l or r <= a:
            return 0
        if a <= l and r <= b:
            if value is not None:
                self._apply(v, l, r, value)
            return self.tree[v]
        self._push(v, l, r)
        m = (l + r) // 2
        answer = self._query(v * 2, l, m, a, b, value)
        answer += self._query(v * 2 + 1, m, r, a, b, value)
        self.tree[v] = self.tree[v * 2] + self.tree[v * 2 + 1]
        return answer
    def add(self, l, r, value):
        if not 0 <= l <= r <= self.n:
            raise IndexError((l, r))
        if l < r:
            self._query(1, 0, self.n, l, r, value)
    def sum(self, l, r):
        if not 0 <= l <= r <= self.n:
            raise IndexError((l, r))
        return self._query(1, 0, self.n, l, r, None) if l < r else 0
```

3.3 Convex Hull Trick [PY]

```
# 직선 최댓값 질의. 단조형은 기울기·질의 x가 증가할 때 분할 상환 O(1).
# Li Chao는 고정 정수 구간에서 임의 순서 질의 O(log 범위).
# 최솟값은 기울기·절편과 결과의 부호를 뒤집는다.
from collections import deque

class MonotoneCHT:
    def __init__(self):
        self.lines = deque()
        self.last_slope = self.last_x = None
    def add(self, a, b, index=None):
        if self.last_slope is not None and a < self.last_slope:
            raise ValueError('slopes must be nondecreasing')
        self.last_slope = a
        q = self.lines
        if q and q[-1][0] == a:
            if q[-1][1] >= b:
                return
            q.pop()
        while len(q) >= 2:
            x, y = q[-2], q[-1]
            if (x[1] - y[1]) * (a - y[0]) < (y[1] - b) * (y[0] - x[0]):
                break
            q.pop()
        q.append((a, b, index))
    def query(self, x):
        if self.last_x is not None and x < self.last_x:
            raise ValueError('query x must be nondecreasing')
        self.last_x = x
        q = self.lines
        if not q:
            raise ValueError('empty hull')
        while len(q) >= 2 and q[0][0] * x + q[0][1] <= q[1][0] * x + q[1][1]:
            q.popleft()
        a, b, index = q[0]
        return a * x + b, index

class LiChao:
    def __init__(self, lo, hi):
        if lo > hi:
```

```
            raise ValueError('empty domain')
        self.lo, self.hi = lo, hi
        self.lines = {}

    def add(self, a, b):
        line = (a, b)
        v, l, r = 1, self.lo, self.hi
        while True:
            if v not in self.lines:
                self.lines[v] = line
                return
            old = self.lines[v]
            m = (l + r) // 2
            left = line[0] * l + line[1] > old[0] * l + old[1]
            middle = line[0] * m + line[1] > old[0] * m + old[1]
            if middle:
                self.lines[v], line = line, old
            if l == r:
                return
            if left != middle:
                v, r = v * 2, m
            else:
                v, l = v * 2 + 1, m + 1

    def query(self, x):
        if not self.lo <= x <= self.hi:
            raise ValueError('x outside domain')
        if not self.lines:
            raise ValueError('empty hull')
        answer = -float('inf')
        v, l, r = 1, self.lo, self.hi
        while v in self.lines:
            a, b = self.lines[v]
            answer = max(answer, a * x + b)
            if l == r:
                break
            m = (l + r) // 2
            if x <= m:
                v, r = v * 2, m
            else:
                v, l = v * 2 + 1, m + 1
        return answer
```

3.4 퍼시스턴트 세그먼트 트리 [PY]

```
# 빈도 트리의 이전 버전을 보존한다. 삽입·k번째 값 O(log N).
# k는 1부터 시작하며 두 버전의 빈도 차는 음수가 아니어야 한다.

class PersistentSegmentTree:
    def __init__(self, n):
        if n <= 0:
            raise ValueError('n must be positive')
        self.n = n
        self.left, self.right, self.count = [0], [0], [0]
        self.roots = [0]

    def insert(self, version, x):
        if not 0 <= x < self.n:
            raise IndexError(x)

        def update(old, l, r):
            v = len(self.count)
            self.left.append(self.left[old])
            self.right.append(self.right[old])
            self.count.append(self.count[old] + 1)
            if r - l > 1:
                m = (l + r) // 2
                if x < m:
                    self.left[v] = update(self.left[old], l, m)
                else:
                    self.right[v] = update(self.right[old], m, r)
            return v

        self.roots.append(update(self.roots[version], 0, self.n))
        return len(self.roots) - 1

    def kth(self, before, after, k):
        a, b = self.roots[before], self.roots[after]
        if not 1 <= k <= self.count[b] - self.count[a]:
            raise ValueError('k outside version difference')
        l, r = 0, self.n
        while r - l > 1:
            m = (l + r) // 2
            count = self.count[self.left[b]] - self.count[self.left[a]]
            if k <= count:
                a, b, r = self.left[a], self.left[b], m
            else:
                k -= count
                a, b, l = self.right[a], self.right[b], m
        return l
```

4. 그래프/트리 (Hard)

4.1 SCC - Kosaraju [PY]

```
# 반복형 코사라주로 강한 연결 요소를 구한다. O(V+E).
# 요소 번호는 축약 그래프의 위상 순서다.

def strongly_connected_components(graph):
    n = len(graph)
    reverse = [[] for _ in graph]
    for v in range(n):
        for u in graph[v]:
            reverse[u].append(v)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        seen[start] = True
        stack = [(start, iter(graph[start]))]
        while stack:
            v, children = stack[-1]
```

```
            u = next(children, None)
            if u is None:
                order.append(v)
                stack.pop()
            elif not seen[u]:
                seen[u] = True
                stack.append((u, iter(graph[u])))
    component = [-1] * n
    groups = []
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = len(groups)
        group, stack = [], [start]
        while stack:
            v = stack.pop()
            group.append(v)
            for u in reverse[v]:
                if component[u] == -1:
                    component[u] = component[start]
                    stack.append(u)
        groups.append(group)
    return component, groups
```

4.2 BCC - Tarjan [PY]

```
# 무향 간선 (u,v)의 단절점·단절선 번호·이중 연결 정점 묶음. O(V+E).
# 다중 간선 허용, 자기 루프 금지. 고립점도 묶음이며 단절점은 중복 소속 가능.

def biconnected_components(n, edges):
    graph = [[] for _ in range(n)]
    for eid, (u, v) in enumerate(edges):
        if u == v:
            raise ValueError('self loops are not supported')
        graph[u].append((v, eid))
        graph[v].append((u, eid))
    tin, low, parent = [-1] * n, [0] * n, [-1] * n
    children = [0] * n
    cuts, bridges, groups, edge_stack = set(), [], [], []
    timer = 0
    for root in range(n):
        if tin[root] != -1:
            continue
        tin[root] = low[root] = timer
        timer += 1
        stack = [(root, -1, iter(graph[root]))]
        while stack:
            v, pe, it = stack[-1]
            item = next(it, None)
            if item is not None:
                u, eid = item
                if eid == pe:
                    continue
                if tin[u] == -1:
                    parent[u] = v
                    children[v] += 1
                    edge_stack.append((v, u, eid))
                    tin[u] = low[u] = timer
                    timer += 1
                    stack.append((u, eid, iter(graph[u])))
                elif tin[u] < tin[v]:
                    low[v] = min(low[v], tin[u])
                    edge_stack.append((v, u, eid))
                continue
            stack.pop()
            p = parent[v]
            if p == -1:
                if children[v] > 1:
                    cuts.add(v)
                if not graph[v]:
                    groups.append([v])
            else:
                low[p] = min(low[p], low[v])
                if low[v] > tin[p]:
                    bridges.append(pe)
                if low[v] >= tin[p]:
                    if parent[p] != -1:
                        cuts.add(p)
                    group = set()
                    while True:
                        a, b, eid = edge_stack.pop()
                        group.update((a, b))
                        if eid == pe:
                            break
                    groups.append(list(group))
    return sorted(cuts), bridges, groups
```

4.3 최대 유량 - Dinic [PY]

```
# 레벨 그래프에서 유량을 흘린다. O(V^2 E), 정수 용량.
# 호출마다 잔여 용량을 갱신하며 추가 유량을 반환한다.
from collections import deque

class Dinic:
    def __init__(self, n):
        self.graph = [[] for _ in range(n)]
        self.to, self.capacity = [], []

    def add_edge(self, u, v, capacity, reverse_capacity=0):
        if min(capacity, reverse_capacity) < 0:
            raise ValueError('negative capacity')
        eid = len(self.to)
        self.graph[u].append(eid)
        self.graph[v].append(eid ^ 1)
        self.to.extend((v, u))
        self.capacity.extend((capacity, reverse_capacity))
        return eid

    def _levels(self, source):
        level = [-1] * len(self.graph)
        level[source] = 0
```

```
        queue = deque([source])
        while queue:
            v = queue.popleft()
            for e in self.graph[v]:
                u = self.to[e]
                if self.capacity[e] > 0 and level[u] == -1:
                    level[u] = level[v] + 1
                    queue.append(u)
        return level

    def max_flow(self, source, sink, limit=None):
        if source == sink:
            raise ValueError('source equals sink')
        if limit is None:
            limit = sum(self.capacity[e] for e in self.graph[source])
        flow = 0
        while flow < limit:
            level = self._levels(source)
            if level[sink] == -1:
                break
            pos = [0] * len(self.graph)
            while flow < limit:
                vertices, path = [source], []
                while vertices and vertices[-1] != sink:
                    v = vertices[-1]
                    while pos[v] < len(self.graph[v]):
                        e = self.graph[v][pos[v]]
                        if self.capacity[e] > 0 and level[self.to[e]] == level[v]
| + 1:
                            break
                        pos[v] += 1
                    if pos[v] == len(self.graph[v]):
                        level[v] = -1
                        vertices.pop()
                        if path:
                            path.pop()
                    else:
                        e = self.graph[v][pos[v]]
                        path.append(e)
                        vertices.append(self.to[e])
                if not vertices:
                    break
                amount = min(limit - flow, min(self.capacity[e] for e in path))
                for e in path:
                    self.capacity[e] -= amount
                    self.capacity[e ^ 1] += amount
                flow += amount
        return flow

    def min_cut(self, source):
        # 유량 제한 없이 최대 유량을 구한 뒤 호출. (시작점 쪽, 반대쪽) 반환.
        level = self._levels(source)
        return ([v for v, d in enumerate(level) if d != -1],
                [v for v, d in enumerate(level) if d == -1])
```

4.4 MCMF [PY]

```
# 최단 증가 경로를 반복해 추가 (유량, 비용)을 구한다. O(A VE), A는 증가 횟수.
# 잔여망을 갱신하며, 도달 가능한 음수 비용 사이클은 거부한다.
from math import inf

class MinCostFlow:
    def __init__(self, n):
        self.n = n
        self.edges = []

    def add_edge(self, u, v, capacity, cost):
        if capacity < 0:
            raise ValueError('negative capacity')
        self.edges.extend(([u, v, capacity, cost], [v, u, 0, -cost]))

    def run(self, source, sink, limit=None):
        if source == sink:
            raise ValueError('source equals sink')
        if limit is None:
            limit = sum(c for u, v, c, w in self.edges if u == source)
        flow = cost = 0
        while flow < limit:
            d, parent = [inf] * self.n, [-1] * self.n
            d[source] = 0
            for step in range(self.n):
                changed = False
                for eid, (u, v, capacity, weight) in enumerate(self.edges):
                    if capacity and d[u] != inf and d[v] > d[u] + weight:
                        d[v], parent[v] = d[u] + weight, eid
                        changed = True
                if not changed:
                    break
                if step == self.n - 1:
                    raise ValueError('reachable negative cost cycle')
            if d[sink] == inf:
                break
            amount, v = limit - flow, sink
            while v != source:
                e = parent[v]
                amount = min(amount, self.edges[e][2])
                v = self.edges[e][0]
            v = sink
            while v != source:
                e = parent[v]
                self.edges[e][2] -= amount
                self.edges[e ^ 1][2] += amount
                v = self.edges[e][0]
            flow += amount
            cost += amount * d[sink]
        return flow, cost
```

4.5 이분 매칭 - Hopcroft Karp [PY]

```
# 왼쪽에서 오른쪽 정점 번호로 향하는 인접 리스트의 최대 이분 매칭.
# 양쪽 매칭 배열과 최소 정점 커버를 반환한다. O((V+E) sqrt(V)).
```

```
from collections import deque

def hopcroft_karp(graph, m):
    n = len(graph)
    left, right = [-1] * n, [-1] * m
    while True:
        distance = [-1] * n
        queue = deque()
        for v in range(n):
            if left[v] == -1:
                distance[v] = 0
                queue.append(v)
        shortest = n + 1
        while queue:
            v = queue.popleft()
            if distance[v] >= shortest:
                continue
            for u in graph[v]:
                w = right[u]
                if w == -1:
                    shortest = distance[v] + 1
                elif distance[w] == -1:
                    distance[w] = distance[v] + 1
                    queue.append(w)
        if shortest == n + 1:
            break
        pos = [0] * n
        for start in range(n):
            if left[start] != -1:
                continue
            stack, links = [start], []
            while stack:
                v = stack[-1]
                if pos[v] == len(graph[v]):
                    distance[v] = -1
                    stack.pop()
                    if links:
                        links.pop()
                    continue
                u = graph[v][pos[v]]
                pos[v] += 1
                w = right[u]
                if w == -1 and distance[v] + 1 == shortest:
                    for a, b in zip(stack, links + [u]):
                        left[a], right[b] = b, a
                    break
                if w != -1 and distance[w] == distance[v] + 1:
                    stack.append(w)
                    links.append(u)
    seen_l = {v for v in range(n) if left[v] == -1}
    seen_r = set()
    queue = deque(seen_l)
    while queue:
        v = queue.popleft()
        for u in graph[v]:
            if u == left[v] or u in seen_r:
                continue
            seen_r.add(u)
            w = right[u]
            if w != -1 and w not in seen_l:
                seen_l.add(w)
                queue.append(w)
    cover = ([v for v in range(n) if v not in seen_l], sorted(seen_r))
    return left, right, cover
```

4.6 최소 공통 조상(LCA) [PY]

```
# 연결된 무향 트리의 최소 공통 조상. 전처리 O(N log N), 질의 O(log N).

class LCA:
    def __init__(self, graph, root=0):
        n = len(graph)
        self.depth = [-1] * n
        self.depth[root] = 0
        parent = [root] * n
        order = [root]
        for v in order:
            for u in graph[v]:
                if self.depth[u] == -1:
                    parent[u] = v
                    self.depth[u] = self.depth[v] + 1
                    order.append(u)
        if len(order) != n or sum(map(len, graph)) != 2 * (n - 1):
            raise ValueError('connected undirected tree required')
        self.up = [parent]
        for _ in range(1, n.bit_length()):
            prev = self.up[-1]
            self.up.append([prev[prev[v]] for v in range(n)])

    def query(self, u, v):
        if self.depth[u] < self.depth[v]:
            u, v = v, u
        diff = self.depth[u] - self.depth[v]
        for k in range(len(self.up)):
            if diff >> k & 1:
                u = self.up[k][u]
        if u == v:
            return u
        for level in reversed(self.up):
            if level[u] != level[v]:
                u, v = level[u], level[v]
        return self.up[0][u]
```

4.7 Heavy Light Decomposition [PY]

```
# 연결된 무향 트리의 경로를 O(log N)개 구간으로 분해한다. 초기화 O(N).
# 구간은 [l,r), 순서 미보장: 교환 가능한 연산에만 사용한다.
# 간선 값은 자식 위치에 저장하고 edges=True로 질의한다.
```

```
class HLD:
    def __init__(self, graph, root=0):
        n = len(graph)
        self.parent = [-1] * n
        self.depth = [0] * n
        self.parent[root] = root
        order = [root]
        for v in order:
            for u in graph[v]:
                if self.parent[u] == -1:
                    self.parent[u] = v
                    self.depth[u] = self.depth[v] + 1
                    order.append(u)
        if len(order) != n or sum(map(len, graph)) != 2 * (n - 1):
            raise ValueError('connected undirected tree required')
        self.size = [1] * n
        heavy = [-1] * n
        for v in reversed(order[1:]):
            p = self.parent[v]
            self.size[p] += self.size[v]
            if heavy[p] == -1 or self.size[v] > self.size[heavy[p]]:
                heavy[p] = v
        self.head, self.pos = [0] * n, [0] * n
        stack, timer = [(root, root)], 0
        while stack:
            v, head = stack.pop()
            while v != -1:
                self.head[v], self.pos[v] = head, timer
                timer += 1
                for u in graph[v]:
                    if self.parent[u] == v and u != v and u != heavy[v]:
                        stack.append((u, u))
                v = heavy[v]
    def path_ranges(self, u, v, edges=False):
        ranges = []
        while self.head[u] != self.head[v]:
            if self.depth[self.head[u]] < self.depth[self.head[v]]:
                u, v = v, u
            ranges.append((self.pos[self.head[u]], self.pos[u] + 1))
            u = self.parent[self.head[u]]
        if self.depth[u] > self.depth[v]:
            u, v = v, u
        l, r = self.pos[u] + int(edges), self.pos[v] + 1
        if l < r:
            ranges.append((l, r))
        return ranges
    def subtree_range(self, v):
        return self.pos[v], self.pos[v] + self.size[v]
```

5. 수학

5.1 나눗셈, 최대공약수, 최소공배수 [PY]

```
# 음수도 처리하는 정수 나눗셈·최대공약수·최소공배수. 나누는 수는 0이 아니어야 한다.
from math import gcd, lcm

def floor_div(a, b):
    return a // b

def ceil_div(a, b):
    return -(-a // b)
```

5.2 빠른 거듭제곱 [PY]

```
# 모듈러 거듭제곱 O(log 지수). 지수는 0 이상, 모듈러스는 1 이상.

def pow_mod(base, exponent, modulus):
    if exponent < 0 or modulus < 1:
        raise ValueError('exponent >= 0 and modulus >= 1 required')
    return pow(base, exponent, modulus)
```

5.3 소수 판별, 소인수분해 [PY]

```
# 제곱근까지 나누어 소인수분해한다. O(sqrt(N)).
from math import isqrt

def is_prime(n):
    return n >= 2 and all(n % p for p in range(2, isqrt(n) + 1))

def factorize(n):
    if n < 1:
        raise ValueError('positive integer required')
    result, p = [], 2
    while p * p <= n:
        count = 0
        while n % p == 0:
            n //= p
            count += 1
        if count:
            result.append((p, count))
        p += 1
    if n > 1:
        result.append((n, 1))
    return result
```

5.4 에라토스테네스의 체, 소인수분해 [PY]

```
# 최소 소인수 표를 만든다. O(N log log N), spf[1]=1.
# 표를 이용한 소인수분해는 O(log x).

def sieve(n):
    spf = list(range(n + 1))
    for p in range(2, int(n ** 0.5) + 1):
        if spf[p] == p:
            for x in range(p * p, n + 1, p):
                if spf[x] == x:
                    spf[x] = p
    return spf
```

```
def factorize(n, spf):
    if not 1 <= n < len(spf):
        raise ValueError('n outside sieve')
    result = []
    while n > 1:
        p, count = spf[n], 0
        while n % p == 0:
            n //= p
            count += 1
        result.append((p, count))
    return result
```

5.5 선형 시간 체, 곱셈적 함수 전처리 [PY]

```
# N까지 소수·최소 소인수·오일러 함수·뫼비우스 함수·약수 개수·합을 구한다. O(N).

def linear_sieve(n):
    spf, phi, mu, tau, sigma = ([0] * (n + 1) for _ in range(5))
    exponent, power = [0] * (n + 1), [0] * (n + 1)
    primes = []
    if n:
        spf[1] = phi[1] = mu[1] = tau[1] = sigma[1] = 1
    for i in range(2, n + 1):
        if spf[i] == 0:
            primes.append(i)
            spf[i], exponent[i], power[i] = i, 1, i
            phi[i], mu[i], tau[i], sigma[i] = i - 1, -1, 2, i + 1
        for p in primes:
            x = i * p
            if x > n:
                break
            spf[x] = p
            if i % p == 0:
                exponent[x] = exponent[i] + 1
                power[x] = power[i] * p
                phi[x] = phi[i] * p
                mu[x] = 0
                tau[x] = tau[i] // (exponent[i] + 1) * (exponent[x] + 1)
                sigma[x] = sigma[i] + power[x] * sigma[i // power[i]]
                break
            exponent[x], power[x] = 1, p
            phi[x], mu[x] = phi[i] * (p - 1), -mu[i]
            tau[x], sigma[x] = tau[i] * 2, sigma[i] * (p + 1)
    return primes, spf, phi, mu, tau, sigma
```

5.6 확장 유클리드 알고리즘 [PY]

```
# ax+by=gcd(a,b)를 만족하는 (g,x,y)를 구한다. 음수·0도 처리한다.

def extended_gcd(a, b):
    old_r, r, old_x, x, old_y, y = a, b, 1, 0, 0, 1
    while r:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_x, x = x, old_x - q * x
        old_y, y = y, old_y - q * y
    sign = -1 if old_r < 0 else 1
    return sign * old_r, sign * old_x, sign * old_y
```

5.7 중국인의 나머지 정리 [PY]

```
# 양의 모듈러스인 합동식을 병합한다. (나머지, 최소공배수), 불가능하면 None.
# 빈 입력은 (0,1). O(K log M).
from math import gcd

def crt(congruences):
    a, m = 0, 1
    for b, n in congruences:
        if n <= 0:
            raise ValueError('positive modulus required')
        g = gcd(m, n)
        if (b - a) % g:
            return None
        reduced = n // g
        t = ((b - a) // g * pow(m // g, -1, reduced)) % reduced
        a += m * t
        m *= reduced
        a %= m
    return a, m
```

5.8 이항 계수를 소수로 나눈 나머지 [PY]

```
# 소수 p에 대한 이항 계수. 전처리 O(p), 질의 O(log_p(n+1)).

class Lucas:
    def __init__(self, p):
        if p < 2:
            raise ValueError('prime p >= 2 required')
        self.p = p
        self.fac = [1] * p
        self.inv = [1] * p
        for i in range(1, p):
            self.fac[i] = self.fac[i - 1] * i % p
        self.inv[-1] = pow(self.fac[-1], p - 2, p)
        for i in range(p - 2, -1, -1):
            self.inv[i] = self.inv[i + 1] * (i + 1) % p
    def calc(self, n, r):
        if r < 0 or n < r:
            return 0
        result, p = 1, self.p
        while n or r:
            a, b = n % p, r % p
            if b > a:
                return 0
            result = result * self.fac[a] * self.inv[b] * self.inv[a - b] % p
            n //= p
            r //= p
        return result
```

5.9 빠른 소수 판별, 소인수분해 - Miller Rabin, Pollard Rho [PY]

```
# 0 이상 2^64 미만의 소수 판별·소인수분해.
```

```
# 밀러 라빈은 확정 판별, 폴라드 로는 최소 소인수 p에 대해 기대 O(sqrt(p)).
from collections import Counter
from math import gcd
from random import randrange

def is_prime(n):
    if n >= 1 << 64:
        raise ValueError('deterministic range is below 2^64')
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % p == 0:
            return n == p
    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1
    for a in (2, 325, 9375, 28178, 450775, 9780504, 1795265022):
        if a % n == 0:
            continue
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True

def _rho(n):
    if n % 2 == 0:
        return 2
    while True:
        x = y = randrange(2, n)
        c, d = randrange(1, n), 1
        while d == 1:
            x = (x * x + c) % n
            y = (y * y + c) % n
            y = (y * y + c) % n
            d = gcd(abs(x - y), n)
        if d != n:
            return d

def factorize(n):
    if not 1 <= n < 1 << 64:
        raise ValueError('1 <= n < 2^64 required')
    factors, stack = [], [n]
    while stack:
        x = stack.pop()
        if x == 1:
            continue
        if is_prime(x):
            factors.append(x)
        else:
            d = _rho(x)
            stack.extend((d, x // d))
    return sorted(Counter(factors).items())
```

5.10 가우스 소거법 - RREF, 랭크, 행렬식, 역행렬 [PY]

```
# 가우스 조던 소거로 (기약 행렬, 계수, 행렬식, 역행렬)을 구한다. O(행^2·열).
# 기본은 정확한 Fraction 연산, mod에는 소수를 지정한다. 입력은 보존한다.
# 정의되지 않는 행렬식·역행렬은 None. 정확한 소수 입력은 Fraction으로 전달한다.
from fractions import Fraction

def gauss(matrix, mod=None):
    n = len(matrix)
    m = len(matrix[0]) if n else 0
    if any(len(row) != m for row in matrix):
        raise ValueError('ragged matrix')
    convert = Fraction if mod is None else lambda x: x % mod
    a = [[convert(x) for x in row] for row in matrix]
    inv = [[convert(i == j) for j in range(n)] for i in range(n)]
    norm = (lambda x: x) if mod is None else lambda x: x % mod
    rank, det = 0, convert(1)
    for col in range(m):
        pivot = next((i for i in range(rank, n) if a[i][col]), None)
        if pivot is None:
            continue
        if pivot != rank:
            a[pivot], a[rank] = a[rank], a[pivot]
            inv[pivot], inv[rank] = inv[rank], inv[pivot]
            det = -det
        value = a[rank][col]
        det = norm(det * value)
        scale = 1 / value if mod is None else pow(value, -1, mod)
        a[rank] = [norm(x * scale) for x in a[rank]]
        inv[rank] = [norm(x * scale) for x in inv[rank]]
        for i in range(n):
            if i == rank:
                continue
            scale = a[i][col]
            a[i] = [norm(x - scale * y) for x, y in zip(a[i], a[rank])]
            inv[i] = [norm(x - scale * y) for x, y in zip(inv[i], inv[rank])]
        rank += 1
        if rank == n:
            break
    determinant = norm(det) if rank == n else 0
    return a, rank, determinant if n == m else None, inv if n == m == rank else No
    | ne
```

5.11 다항식 곱셈(FFT) [PY]

```
# FFT·NTT·비트 합성곱 O(N log N). FFT는 큰 계수에서 반올림 오차가 생길 수 있다.
# multiply_mod는 정확한 NTT/CRT, 범위 초과 시 O(len(a)·len(b)) 직접 곱셈.
# 뒤쪽 0을 포함한 len(a)+len(b)-1개 계수를 반환한다.
from cmath import exp, pi
```

```
def transform(a, inverse=False, mod=None, root=3):
    n = len(a)
    if not n or n & (n - 1):
        raise ValueError('nonempty power of two required')
    if mod is not None and (mod - 1) % n:
        raise ValueError('transform length must divide mod-1')
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j ^= bit
        if i < j:
            a[i], a[j] = a[j], a[i]
    length = 2
    while length <= n:
        if mod is None:
            step = exp(((-2 if inverse else 2) * pi * 1j) / length)
        else:
            step = pow(root, (mod - 1) // length, mod)
            if inverse:
                step = pow(step, -1, mod)
        for start in range(0, n, length):
            w = 1
            for j in range(start, start + length // 2):
                u, v = a[j], a[j + length // 2] * w
                a[j], a[j + length // 2] = u + v, u - v
                w *= step
                if mod is not None:
                    a[j] %= mod
                    a[j + length // 2] %= mod
                    w %= mod
        length *= 2
    if inverse:
        scale = 1 / n if mod is None else pow(n, -1, mod)
        for i in range(n):
            a[i] = a[i] * scale if mod is None else a[i] * scale % mod

def multiply(a, b, mod=None, root=3):
    if not a or not b:
        return []
    size = len(a) + len(b) - 1
    n = 1 << (size - 1).bit_length()
    a, b = list(a) + [0] * (n - len(a)), list(b) + [0] * (n - len(b))
    transform(a, mod=mod, root=root)
    transform(b, mod=mod, root=root)
    a = [x * y if mod is None else x * y % mod for x, y in zip(a, b)]
    transform(a, True, mod, root)
    return [round(x.real) for x in a[:size]] if mod is None else a[:size]

def multiply_mod(a, b, mod=998244353, root=3):
    if mod <= 0:
        raise ValueError('positive modulus required')
    if not a or not b:
        return []
    a, b = [x % mod for x in a], [x % mod for x in b]
    size = len(a) + len(b) - 1
    n = 1 << (size - 1).bit_length()
    primes = (167772161, 469762049, 1224736769)
    if mod == 998244353 and n <= 1 << 23:
        return multiply(a, b, mod, root)
    p, q, r = primes
    if n <= 1 << 24 and min(len(a), len(b)) * max(a) * max(b) < p * q * r:
        parts = [multiply(a, b, prime, 3) for prime in primes]
        inv_p, inv_pq = pow(p, -1, q), pow(p * q, -1, r)
        result = []
        for x, y, z in zip(*parts):
            value = x + (y - x) * inv_p % q * p
            value += (z - value) * inv_pq % r * p * q
            result.append(value % mod)
        return result
    result = [0] * size
    for i, x in enumerate(a):
        for j, y in enumerate(b):
            result[i + j] = (result[i + j] + x * y) % mod
    return result

def bitwise_convolution(a, b, kind='xor'):
    # 같은 2의 거듭제곱 길이를 가진 정수 배열의 정확한 비트 합성곱.
    n = len(a)
    if n != len(b) or not n or n & (n - 1) or kind not in ('xor', 'or', 'and'):
        raise ValueError('invalid length or transform kind')
    def fwht(values, inverse=False):
        step = 1
        while step < n:
            for base in range(0, n, step * 2):
                for j in range(base, base + step):
                    u, v = values[j], values[j + step]
                    if kind == 'xor':
                        values[j], values[j + step] = u + v, u - v
                    elif kind == 'or':
                        values[j + step] = v - u if inverse else v + u
                    else:
                        values[j] = u - v if inverse else u + v
            step *= 2
        if inverse and kind == 'xor':
            values[:] = [x // n for x in values]
    a, b = list(a), list(b)
    fwht(a)
    fwht(b)
    result = [x * y for x, y in zip(a, b)]
    fwht(result, True)
    return result
```

6. 문자열

6.1 문자열 해싱 [PY]

```
# 구간 [l,r) 해시. 전처리 O(N), 질의 O(1).
# 충돌 가능: 실제 문자열을 확인하거나 모듈러스를 두 개 사용한다.

class Hashing:
    def __init__(self, text, base=917, mod=998244353):
        self.mod = mod
        self.h, self.p = [0], [1]
        for char in text:
            value = ord(char) if isinstance(char, str) else char
            self.h.append((self.h[-1] * base + value) % mod)
            self.p.append(self.p[-1] * base % mod)

    def get(self, l, r):
        if not 0 <= l <= r < len(self.h):
            raise IndexError((l, r))
        return (self.h[r] - self.h[l] * self.p[r - l]) % self.mod
```

6.2 문자열 매칭 - KMP [PY]

```
# 접두사 실패 함수를 이용한 패턴 검색 O(N+M). 일치 시작 위치를 반환한다.
# 빈 패턴은 모든 N+1개 경계에 일치한다.

def prefix_function(pattern):
    fail = [0] * len(pattern)
    for i in range(1, len(pattern)):
        j = fail[i - 1]
        while j and pattern[i] != pattern[j]:
            j = fail[j - 1]
        if pattern[i] == pattern[j]:
            j += 1
        fail[i] = j
    return fail

def kmp(text, pattern):
    if not pattern:
        return list(range(len(text) + 1))
    fail, result, j = prefix_function(pattern), [], 0
    for i, char in enumerate(text):
        while j and char != pattern[j]:
            j = fail[j - 1]
        if char == pattern[j]:
            j += 1
        if j == len(pattern):
            result.append(i - j + 1)
            j = fail[j - 1]
    return result
```

6.3 가장 긴 팰린드롬 부분 문자열 - Manacher [PY]

```
# 구분자를 넣은 2N+1개 중심의 팰린드롬 반지름. O(N).
# 반지름은 원문 팰린드롬 길이이며 구분자는 입력 문자와 충돌하지 않는다.

def manacher(text):
    separator = object()
    values = [separator]
    for char in text:
        values.extend((char, separator))
    radius = [0] * len(values)
    center = right = 0
    for i in range(len(values)):
        if i < right:
            radius[i] = min(right - i, radius[2 * center - i])
        while (i - radius[i] > 0 and i + radius[i] + 1 < len(values)
               and values[i - radius[i] - 1] == values[i + radius[i] + 1]):
            radius[i] += 1
        if i + radius[i] > right:
            center, right = i, i + radius[i]
    return radius
```

6.4 문자열 매칭 - Z [PY]

```
# 각 위치에서 전체 문자열의 접두사와 일치하는 길이. O(N), z[0]=N.

def z_function(text):
    n = len(text)
    z = [0] * n
    if n:
        z[0] = n
    l = r = 0
    for i in range(1, n):
        if i < r:
            z[i] = min(r - i, z[i - l])
        while i + z[i] < n and text[z[i]] == text[i + z[i]]:
            z[i] += 1
        if i + z[i] > r:
            l, r = i, i + z[i]
    return z
```

6.5 접미사 배열 [PY]

```
# 접미사 배열과 인접 접미사의 공통 접두사 길이. O(N log N).
# lcp[i]=LCP(sa[i-1],sa[i]), lcp[0]=0. 인덱스는 0부터 시작한다.

def suffix_array(text):
    n = len(text)
    if not n:
        return [], []
    alphabet = {c: i + 1 for i, c in enumerate(sorted(set(text)))}
    rank = [alphabet[c] for c in text]
    sa, step = list(range(n)), 1

    def counting_sort(indices, keys, size):
        count = [0] * (size + 1)
        for i in indices:
            count[keys[i]] += 1
        for i in range(1, len(count)):
            count[i] += count[i - 1]
        result = [0] * len(indices)
        for i in reversed(indices):
            count[keys[i]] -= 1
            result[count[keys[i]]] = i
```

```
        return result
    while True:
        second = [rank[i + step] if i + step < n else 0 for i in range(n)]
        size = max(rank)
        sa = counting_sort(counting_sort(sa, second, size), rank, size)
        new, classes = [0] * n, 1
        new[sa[0]] = classes
        for a, b in zip(sa, sa[1:]):
            if (rank[a], second[a]) != (rank[b], second[b]):
                classes += 1
            new[b] = classes
        rank = new
        if classes == n:
            break
        step *= 2
    lcp, length = [0] * n, 0
    for i in range(n):
        pos = rank[i] - 1
        if pos == 0:
            length = 0
            continue
        j = sa[pos - 1]
        while i + length < n and j + length < n and text[i + length] == text[j + l
    | ength]:
            length += 1
        lcp[pos] = length
        length = max(0, length - 1)
    return sa, lcp
```

7. 계산 기하

7.1 2차원 계산 기하 템플릿 + CCW [PY]

```
# 정수 점 (x,y)의 내적·외적·방향·거리 제곱. O(1).
# 실수 좌표를 쓰면 오차 허용 기준을 별도로 정해야 한다.

def add(a, b):
    return a[0] + b[0], a[1] + b[1]

def sub(a, b):
    return a[0] - b[0], a[1] - b[1]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def orient(a, b, c):
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

def ccw(a, b, c):
    value = orient(a, b, c)
    return (value > 0) - (value < 0)

def distance_squared(a, b):
    return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2
```

7.2 360도 각도 정렬 [PY]

```
# 원점 우선, +x축부터 반시계 방향 정렬. 같은 각도는 가까운 점 우선. O(N log N).
from functools import cmp_to_key

def polar_sort(points):
    def half(p):
        return 0 if p[1] > 0 or p[1] == 0 and p[0] >= 0 else 1

    def compare(a, b):
        if a == (0, 0) or b == (0, 0):
            return (b == (0, 0)) - (a == (0, 0))
        if half(a) != half(b):
            return half(a) - half(b)
        cross = a[0] * b[1] - a[1] * b[0]
        if cross:
            return -1 if cross > 0 else 1
        da, db = a[0] ** 2 + a[1] ** 2, b[0] ** 2 + b[1] ** 2
        return (da > db) - (da < db)

    return sorted(points, key=cmp_to_key(compare))
```

7.3 다각형 넓이 [PY]

```
# 경계 순서로 주어진 단순 다각형의 넓이 두 배를 정수로 구한다. O(N).

def polygon_area2(points):
    return abs(sum(points[i][0] * points[(i + 1) % len(points)][1]
                   - points[i][1] * points[(i + 1) % len(points)][0]
                   for i in range(len(points))))
```

7.4 선분 교차 판정 [PY]

```
# 선분 교차 O(1). (0,None): 없음, (1,점): 한 점, (-1,None): 구간 겹침.
# 교점은 Fraction으로 정확히 계산하며 길이 0인 선분도 처리한다.
from fractions import Fraction

def segment_intersection(a, b, c, d):
    def cross(u, v):
        return u[0] * v[1] - u[1] * v[0]

    def sub(u, v):
        return u[0] - v[0], u[1] - v[1]

    u, v, w = sub(b, a), sub(d, c), sub(c, a)
    det = cross(u, v)
    if det:
        t, s = Fraction(cross(w, v), det), Fraction(cross(w, u), det)
        if 0 <= t <= 1 and 0 <= s <= 1:
            return 1, (a[0] + t * u[0], a[1] + t * u[1])
        return 0, None
    if cross(u, w) or cross(v, w):
        return 0, None
    lo, hi = max(min(a, b), min(c, d)), min(max(a, b), max(c, d))
    if lo > hi:
        return 0, None
    return (1, lo) if lo == hi else (-1, None)
```

7.5 다각형 내부 판별 [PY]

```
# 단순 다각형 내부 판별 O(N). 감김 수를 사용하며 경계를 포함한다.

def point_in_polygon(points, p):
    winding = 0
    for i, a in enumerate(points):
        b = points[(i + 1) % len(points)]
        cross = (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0])
        if cross == 0 and min(a, b) <= p <= max(a, b):
            return True
        if a[1] <= p[1] < b[1] and cross > 0:
            winding += 1
        if b[1] <= p[1] < a[1] and cross < 0:
            winding -= 1
    return winding != 0
```

7.6 볼록 껍질 - Monotone Chain [PY]

```
# 단조 체인 볼록 껍질 O(N log N). 반시계 방향, 끝점 중복·일직선 내부점 제거.

def convex_hull(points):
    points = sorted(set(points))
    if len(points) <= 1:
        return points

    def orient(a, b, c):
        return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

    def chain(items):
        hull = []
        for p in items:
            while len(hull) > 1 and orient(hull[-2], hull[-1], p) <= 0:
                hull.pop()
            hull.append(p)
        return hull

    return chain(points)[:-1] + chain(reversed(points))[:-1]
```

7.7 가장 먼 두 점 - Rotating Calipers [PY]

```
# 반시계 방향 볼록 껍질에서 가장 먼 두 점을 구한다. O(N).
# 끝점 중복·일직선 내부점은 제거해야 한다. 빈 입력은 None.

def calipers(hull):
    n = len(hull)
    if not n:
        return None
    if n <= 2:
        return hull[0], hull[-1]

    def area(i, k, j):
        a, b, c = hull[i], hull[k], hull[j]
        return abs((b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0]))

    best, pair, j = -1, None, 1
    for i in range(n):
        k = (i + 1) % n
        while area(i, k, (j + 1) % n) > area(i, k, j):
            j = (j + 1) % n
        for a, b in ((hull[i], hull[j]), (hull[k], hull[j])):
            distance = (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2
            if distance > best:
                best, pair = distance, (a, b)
    return pair
```

7.8 볼록 다각형 내부 판별 [PY]

```
# 반시계 볼록 다각형 내부 판별 O(log N), 경계 포함.
# 끝점 중복·일직선 내부점 제거. 정점이 0~2개인 경우도 처리한다.

def point_in_convex_polygon(hull, p):
    def orient(a, b, c):
        return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

    def on_segment(a, b):
        return orient(a, b, p) == 0 and min(a, b) <= p <= max(a, b)

    n = len(hull)
    if n < 3:
        return bool(n) and on_segment(hull[0], hull[-1])
    a = hull[0]
    first, last = orient(a, hull[1], p), orient(a, hull[-1], p)
    if first < 0 or last > 0:
        return False
    if first == 0:
        return on_segment(a, hull[1])
    if last == 0:
        return on_segment(a, hull[-1])
    l, r = 1, n - 1
    while r - l > 1:
        m = (l + r) // 2
        if orient(a, hull[m], p) >= 0:
            l = m
        else:
            r = m
    return orient(hull[l], hull[r], p) >= 0
```

8. 기타

8.1 가장 긴 증가하는 부분 수열(LIS) [PY]

```
# 엄격히 증가하는 최장 부분 수열 O(N log N).
# 같은 값도 허용하려면 bisect_left를 bisect_right로 바꾼다.
from bisect import bisect_left

def lis(values):
    tails, ends, previous = [], [], [-1] * len(values)
    for i, value in enumerate(values):
        pos = bisect_left(tails, value)
        if pos:
            previous[i] = ends[pos - 1]
        if pos == len(tails):
            tails.append(value)
            ends.append(i)
        else:
            tails[pos], ends[pos] = value, i
```

```
    result = []
    i = ends[-1] if ends else -1
    while i != -1:
        result.append(values[i])
        i = previous[i]
    return result[::-1]
```

8.2 이분 탐색 [PY]

```
# 단조 조건의 경계를 구한다. 구간 [lo,hi), 조건 평가 O(log 범위).
def first_true(lo, hi, check):
    # [lo,hi)의 거짓→참 경계. 첫 참의 위치, 없으면 hi.
    while lo < hi:
        mid = (lo + hi) // 2
        if check(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

def last_true(lo, hi, check):
    # [lo,hi)의 참→거짓 경계. 마지막 참의 위치, 없으면 lo-1.
    return first_true(lo, hi, lambda x: not check(x)) - 1
```

8.3 삼분 탐색 [PY]

```
# 단봉 함수의 최솟값. 닫힌 구간 [lo,hi], 정수는 가장 작은 최소 위치 반환.
# 최소 구간 밖에서 엄격히 감소·증가해야 한다. 정수 O(log 범위).

def ternary_integer(lo, hi, f):
    if lo > hi:
        raise ValueError('empty interval')
    while hi - lo > 3:
        a = lo + (hi - lo) // 3
        b = hi - (hi - lo) // 3
        if f(a) > f(b):
            lo = a + 1
        else:
            hi = b - 1
    return min(range(lo, hi + 1), key=f)

def ternary_real(lo, hi, f, iterations=100):
    for _ in range(iterations):
        a, b = (2 * lo + hi) / 3, (lo + 2 * hi) / 3
        if f(a) > f(b):
            lo = a
        else:
            hi = b
    return (lo + hi) / 2
```

8.4 Python 랜덤, 순서 통계, 비트마스킹 [PY]

```
# 정렬 리스트의 순위·k번째 조회 O(log N)·O(1), 삽입·삭제 O(N).
# 갱신이 많으면 좌표 압축과 펜윅 트리·세그먼트 트리를 사용한다.
from bisect import bisect_left

class OrderedSet:
    def __init__(self, values=()):
        self.values = sorted(set(values))

    def add(self, value):
        i = bisect_left(self.values, value)
        if i == len(self.values) or self.values[i] != value:
            self.values.insert(i, value)

    def discard(self, value):
        i = bisect_left(self.values, value)
        if i < len(self.values) and self.values[i] == value:
            self.values.pop(i)

    def order_of_key(self, value):
        return bisect_left(self.values, value)

    def find_by_order(self, index):
        return self.values[index]

def trailing_zeros(x):
    if x <= 0:
        raise ValueError('positive integer required')
    return (x & -x).bit_length() - 1

def leading_zeros(x, width=64):
    if not 0 <= x < 1 << width:
        raise ValueError('value outside unsigned width')
    return width - x.bit_length()

def next_same_popcount(x):
    # 1인 비트 수가 같은, 현재 값보다 큰 최소 양의 정수.
    if x <= 0:
        raise ValueError('positive integer required')
    low = x & -x
    ripple = x + low
    return ripple | ((ripple ^ x) // low >> 2)

def digit_frequency(n, digit):
    # [1,n]에서 십진 숫자의 등장 횟수. 앞쪽 0 제외, O(log n).
    if not 0 <= digit <= 9:
        raise ValueError('invalid digit')
    answer, place = 0, 1
    while place <= n:
        high, current, low = n // (place * 10), n // place % 10, n % place
        if digit:
            answer += high * place
        elif high:
            answer += (high - 1) * place
        if digit or high:
            if current > digit:
                answer += place
            elif current == digit:
                answer += low + 1
        place *= 10
    return answer
```

8.5 빠른 입력(Fast Input from STDIN) [PY]

```
# 공백으로 구분한 정수 입출력. 전체 읽기는 빠르고, 분할 읽기는 메모리를 절약한다.
import sys

def read_ints(stream=None):
    if stream is None:
        stream = sys.stdin.buffer
    return list(map(int, stream.read().split()))

def iter_ints(stream=None, chunk_size=1 << 16):
    # 입력은 공백으로 구분한 올바른 십진 정수여야 한다.
    if stream is None:
        stream = sys.stdin.buffer
    if chunk_size <= 0:
        raise ValueError('positive chunk size required')
    value, sign, active = 0, 1, False
    while True:
        chunk = stream.read(chunk_size)
        if not chunk:
            break
        for c in chunk:
            if 48 <= c <= 57:
                value = value * 10 + c - 48
                active = True
            elif c == 45:
                sign = -1
            else:
                if active:
                    yield sign * value
                value, sign, active = 0, 1, False
    if active:
        yield sign * value

def write_lines(values, stream=None):
    if stream is None:
        stream = sys.stdout
    stream.write('\n'.join(map(str, values)))
```

8.6 구간별 약수 최대 개수, 최대 소수

원본의 수치표를 보존했습니다. 각 행의 지수는 오른쪽 소수 목록에 대응합니다.

< 10^k	number	divisors	2 3 5 71113171923293137
1	6	4	1 1
2	60	12	2 1 1
3	840	32	3 1 1 1
4	7560	64	3 3 1 1
5	83160	128	3 3 1 1 1
6	720720	240	4 2 1 1 1 1
7	8648640	448	6 3 1 1 1 1
8	73513440	768	5 3 1 1 1 1 1
9	735134400	1344	6 3 2 1 1 1 1
10	6983776800	2304	5 3 2 1 1 1 1 1
11	97772875200	4032	6 3 2 2 1 1 1 1
12	963761198400	6720	6 4 2 1 1 1 1 1 1
13	9316358251200	10752	6 3 2 1 1 1 1 1 1 1
14	97821761637600	17280	5 4 2 2 1 1 1 1 1 1
15	866421317361600	26880	6 4 2 1 1 1 1 1 1 1 1
16	8086598962041600	41472	8 3 2 2 1 1 1 1 1 1 1
17	74801040398884800	64512	6 3 2 2 1 1 1 1 1 1 1 1
18	897612484786617600	103680	8 4 2 2 1 1 1 1 1 1 1 1

< 10^k	prime	# of prime	< 10^k	prime
1	7	4	10	9999999967
2	97	25	11	99999999977
3	997	168	12	999999999989
4	9973	1229	13	9999999999971
5	99991	9592	14	99999999999973
6	999983	78498	15	999999999999989
7	9999991	664579	16	9999999999999937
8	99999989	5761455	17	99999999999999997
9	999999937	50847534	18	999999999999999989

8.7 카탈란 수, 심슨 적분, 그런디 정리, 픽의 정리, 페르마 포인트, 오일러 정리

```
# 바닥 함수 합·카탈란 수·수치 적분·근 찾기·집합 분할 수.
from math import comb

def floor_sum(n, m, a, b):
    if n < 0 or m <= 0:
        raise ValueError('n >= 0 and m > 0 required')
    answer = 0
    while True:
        q, a = divmod(a, m)
        answer += q * n * (n - 1) // 2
        q, b = divmod(b, m)
        answer += q * n
        top = a * n + b
        if top < m:
            return answer
        n, b = divmod(top, m)
        m, a = a, m

def catalan(n):
    return comb(2 * n, n) // (n + 1)
```

```
def simpson(f, a, b, steps=1000):
    if steps <= 0 or steps % 2:
        raise ValueError('positive even steps required')
    h = (b - a) / steps
    return h / 3 * (f(a) + f(b) + sum((4 if i % 2 else 2) * f(a + i * h)
                                      for i in range(1, steps)))

def newton(f, derivative, x, tolerance=1e-12, iterations=100):
    # 뉴턴법으로 근을 찾는다. 수렴 실패 가능: 결과의 잔차를 확인한다.
    for _ in range(iterations):
        value = f(x)
        if abs(value) <= tolerance:
            return x
        slope = derivative(x)
        if slope == 0:
            raise ValueError('zero derivative')
        x -= value / slope
    if abs(f(x)) <= tolerance:
        return x
    raise ValueError('iteration limit reached')

def stirling2_row(n, k):
    row = [1] + [0] * k
    for _ in range(n):
        for j in range(k, 0, -1):
            row[j] = j * row[j] + row[j - 1]
        row[0] = 0
    return row

def bell(n):
    return sum(stirling2_row(n, n))

def derangements(n):
    a, b = 1, 0
    if n == 0:
        return a
    for i in range(2, n + 1):
        a, b = b, (i - 1) * (a + b)
    return b
```

카탈란 수: C(n) = comb(2*n,n) // (n+1). 처음 값은 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900. 길이 2n의 올바른 괄호 문자열, 잎 n+1개의 순서 있는 full binary tree, 볼록 (n+2)각형의 삼각 분할을 센다.

심슨 공식: n은 양의 짝수, h=(b-a)/n, x_i=a+i*h. 적분 근삿값은 h/3 * (f(x_0)+f(x_n)+4*sum(f(x_i), i 홀수) +2*sum(f(x_i), 0<i<n 짝수)). f의 4계 도함수가 연속이고 절댓값이 M 이하이면 절대 오차는 M*(b-a)*h^4/180 이하.

Nim: 일반 규칙에서는 모든 더미 크기의 XOR가 0이면 후공 승리, 아니면 선공 승리. Grundy 값은 가능한 다음 상태의 값들의 mex(포함되지 않은 최소 비음수 정수). 독립적인 게임의 합은 각 Grundy 값의 XOR. 유한 비순환 impartial normal-play 게임에 적용한다.

Subtraction game: 한 번에 한 더미에서 1..k개를 제거할 때 각 더미 크기를 k+1로 나눈 나머지가 Grundy 값이다. 이 값들의 XOR로 판단한다.

Index-k Nim (Moore's Nim): 한 번에 1..k개 더미를 고르고 각 더미에서 양의 개수를 제거한다. 각 이진 자릿수에서 1의 개수를 k+1로 나눈 나머지가 모두 0인 상태가 패배 상태다. 이 나머지 벡터를 일반적인 Grundy 값으로 취급하지 않는다.

Misere Nim: 마지막 돌을 가져가는 쪽이 패배한다. 0인 더미를 제외하고 모두 크기 1이면 양의 더미 수가 홀수일 때 후공 승리. 크기 2 이상 더미가 있으면 XOR가 0일 때 후공 승리. 빈 시작 상태는 게임 규칙의 종료 정의를 확인한다.

Pick 정리: 단순 격자 다각형에서 넓이 A = 내부 격자점 수 I + 경계 격자점 수 B/2 - 1. 경계 격자점 수는 각 변의 gcd(abs(dx), abs(dy)) 합이다. floor_sum(n,m,a,b)는 i=0..n-1에 대해 floor((a*i+b)/m)를 합한다. 아래 Reference.py에 반복형 구현을 포함했다.

페르마 포인트: 삼각형 꼭짓점까지 거리 합을 최소화한다. 120도 이상인 각이 있으면 그 꼭짓점이다. 모든 각이 120도 미만이면 각 변 바깥쪽 정삼각형의 새 꼭짓점과 반대 꼭짓점을 잇는 선분들이 만나는 점이다. 최소 거리 합은 sqrt((a*a+b*b+c*c+4*sqrt(3)*A)/2), A는 삼각형 넓이.

오일러 정리: gcd(a,m)=1이면 a^phi(m) = 1 (mod m). 이 경우 지수를 phi(m)로 나눠 줄일 수 있다. 서로소가 아닐 때 무조건 지수를 줄이면 틀린다. 충분한 지수 e >= ceil(log2(m)), m>=2에서는 a^e = a^(e % phi(m) + phi(m)) (mod m)를 사용할 수 있다. 작은 지수는 원래대로 계산한다.

소수 p에 대해 g^0+...+g^(p-2)는 g=1 (mod p)이면 -1 (mod p), g가 0도 1도 아니면 0 (mod p)이다. g=0이면 합은 1이므로 별도 처리한다.

8.8 경우의 수 - 포함 배제, 스털링 수, 벨 수

구별되는 n개 공을 구별되는 k개 상자에 모두 비지 않게 넣는 경우(전사 함수): sum((-1)^(k-i)*comb(k,i)*i^n, i=0..k) = k!*S2(n,k). 원문 첫 항목의 '공 구별 X'를 '공 구별 O'로 정정했다. 구별되지 않는 공, 구별되는 양의 상자라면 comb(n-1,k-1)이다.

구별되는 공, 구별되지 않는 k개 비지 않은 상자: 제2종 스털링 수 S2(n,k)=k*S2(n-1,k)+S2(n-1,k-1). S2(0,0)=1, 유효 범위 밖은 0. 포함 배제식은 위 전사 함수 수를 k!로 나눈 값이다. 모듈러 나눗셈에는 역원의 존재 조건을 확인한다.

벨 수: B(n)=sum(S2(n,k), k=0..n). 최대 k개 블록이면 B(n,k)=sum(S2(n,j), j=0..k). B(n)=sum(comb(n-1,i)*B(i), i=0..n-1), B(0)=1. 최대 k개 식은 sum(i^n/i! * sum((-1)^j/j!, j=0..k-i), i=0..k)로도 정리된다. 이 식의 나눗셈은 유리수 또는 유효한 모듈러 역원으로 계산한다.

완전 순열: D(0)=1, D(1)=0, D(n)=(n-1)*(D(n-1)+D(n-2)).

부호 없는 제1종 스털링 수: c(n,k)=(n-1)*c(n-1,k)+c(n-1,k-1). 부호 있는 제1종 스털링 수: s(n,k)=s(n-1,k-1)-(n-1)*s(n-1,k). 둘 다 (0,0)=1. 원문의 signed 점화식 부호를 정정했다.

정수 n을 정확히 k개의 양의 정수로 분할: p(n,k)=p(n-1,k-1)+p(n-k,k), p(0,0)=1. 전체 분할 수: p(0)=1, p(n)=sum((-1)^(k-1)*(p(n-k*(3*k-1)//2)+p(n-k*(3*k+1)//2)), k>=1; 음수 인자의 p는 0. 원문의 부호를 정정했다.

카탈란 등가식: C(n)=comb(2*n,n)-comb(2*n,n+1)=(2*n)!/(n!*(n+1)!). C(0)=1, C(n)=sum(C(i)*C(n-1-i), i=0..n-1) for n>=1.

8.9 삼각형의 오심 - 외심, 내심, 무게중심, 수심, 방심

비퇴화 삼각형에서 변 a,b,c는 각각 꼭짓점 A,B,C의 맞은편 길이. 반둘레 p=(a+b+c)/2. 넓이 K=sqrt(p*(p-a)*(p-b)*(p-c)). 외접 반지름 R=a*b*c/(4*K), 내접 반지름 r=K/p.

중선 m_a=sqrt(2*b*b+2*c*c-a*a)/2. 각 이등분선 길이 t_a=sqrt(b*c*(1-(a/(b+c))^2)). 사인 법칙 sin(A)/a=1/(2*R). 코사인 법칙 a*a=b*b+c*c-2*b*c*cos(A). 탄젠트 법칙 (a+b)/(a-b)=tan((A+B)/2)/tan((A-B)/2), 분모가 0이면 별도 처리.

무게 (alpha,beta,gamma)에 대응하는 점은 (alpha*A+beta*B+gamma*C)/(alpha+beta+gamma). X=b*b+c*c-a*a, Y=a*a+c*c-b*b, Z=a*a+b*b-c*c라 두면 외심의 무게는 (a*a*X,b*b*Y,c*c*Z), 내심은 (a,b,c), 무게중심은 (1,1,1), 수심은 (Y*Z,Z*X,X*Y), A에 대응하는 방심은 (-a,b,c). 직각 삼각형의 수심 공식은 0/0이 될 수 있으므로 직각 꼭짓점을 직접 반환한다.

8.10 미적분, 뉴턴 랩슨법

실수 정의역에서 (arcsin x)'=1/sqrt(1-x*x), (arccos x)'=-1/sqrt(1-x*x), (arctan x)'=1/(1+x*x), (tan x)'=1+tan(x)^2.

적분 tan(a*x) dx = -ln(abs(cos(a*x)))/a + C. 적분 x*sin(a*x) dx = (sin(a*x)-a*x*cos(a*x))/(a*a)+C. 두 식 모두 a != 0.

Newton: x_next=x-f(x)/f'(x). 초기 추정치와 도함수에 따라 발산·순환할 수 있다. 잔차, 도함수 0, 반복 상한을 확인한다. 이분 탐색처럼 수렴이 보장되는 방법과 혼용할 수 있다.

Green 정리: 경계 C에서 integral(L dx + M dy) = 영역 D에서 double_integral(dM/dx - dL/dy) dx dy. C는 양의 방향의 조각별 매끄러운 단순 폐곡선, L,M은 D를 포함하는 열린 영역에서 연속 편도함수를 가져야 한다.

8.11 문제 풀이 체크리스트

비슷한 문제를 풀어본 적이 있는가? 단순한 완전 탐색에서 시작할 수 있는가? 예제를 직접 풀고 그 과정을 수식화할 수 있는가? 문제를 단순화하거나 그림으로 표현할 수 있는가?

문제를 작은 부분으로 분해할 수 있는가? 뒤에서부터 풀 수 있는가? 순서를 강제할 수 있는가? 답을 특정한 정규형으로 제한할 수 있는가?

구간을 통째로 선택하는 문제는 플로우와 구간 자료구조를 고려한다. 원문 힌트 (i,i+1,k,0), (s,e,1,w), (N,T,k,0)는 (시작,끝,용량,비용) 간선 예시다. 모델과 비용 부호는 문제에 맞춰 정한다.

a=b를 만족시켜야 한다면 a만 변경, b만 변경, 둘 다 변경, 역방향 변환을 각각 생각한다. 당연하다고 가정한 내용을 다시 점검한다.

확률 문제는 DP와 이분 탐색을 고려한다(원문 예: NYPC 2019 Finals C). 최대·최소 문제는 이분 탐색, 그리디의 교환 논증·접두사 고정, 순서 고정 DP를 검토한다.

Python 제출 전: 입력량과 메모리, 정수 연산 비용, 0-based/[l,r) 경계, 빈 입력·중복·연결되지 않은 그래프, 음수 가중치, 재귀 사용 여부, float 오차, 해시 충돌을 확인한다. 로컬 테스트 통과와 온라인 저지 통과를 구분한다.